

[项目实战]在线教育-数仓搭建阶段(实时)

练习题

习题一 编写 ClickHouseUtil

请尝试使用 GPT 编写 ClickHouseUtil。

提醒 编写 Flink 代码

截至目前 GPT 较难写出准确的 Flink 代码，尤其是业务逻辑过长时。

习题二 编写接口 Demo

使用 GPT 编写整个 Spring boot 接口项目是很困难的，但是我们可以让 GPT 给出各种 demo。辅助我们加快开发进度。请尝试使用 GPT 编写 Spring boot 接口的 demo。

参考答案

习题一 编写工具类

第 1 轮提示

提示

我正在用 java 写一个 flink 项目，现在需要一个用来生成 clickhouse jdbc connector 的工具类，请尽可能使用 flink 提供的 api。

在创建 clickhouse jdbc connector 的时候，我希望动态指定 sql。另外，后面我写入数据的时候，希望能通过反射的方式获取这个对象的所有属性，并判断它有没有 TransientSink 注解，如果有这个注解的话，就忽略这个属性

答

首先，确保你的项目已经添加了 Flink Connector 和 ClickHouse JDBC 的依赖。

在 Maven pom.xml 文件中添加以下依赖：

```
<dependencies>
  <dependency>
    <groupId>org.apache.flink</groupId>
    <artifactId>flink-connector-jdbc_2.11</artifactId>
    <version>1.12.0</version>
  </dependency>
  <dependency>
    <groupId>ru.yandex.clickhouse</groupId>
    <artifactId>clickhouse-jdbc</artifactId>
    <version>0.2.4</version>
  </dependency>
  <!-- 其他的依赖 -->
</dependencies>
```

接下来，可以创建一个帮助类来创建 ClickHouse JDBC Connector。考虑到你可能需要动态地指定 SQL，可以把 SQL 作为创建方法的一个参数：

```
import org.apache.flink.connector.jdbc.JdbcConnectionOptions;
import org.apache.flink.connector.jdbc.JdbcExecutionOptions;
import org.apache.flink.connector.jdbc.JdbcSink;
import org.apache.flink.streaming.api.functions.sink.SinkFunction;

public class ClickHouseJdbcSinkFactory {
    private static final String CLICKHOUSE_DRIVER_NAME =
"ru.yandex.clickhouse.ClickHouseDriver";

    public static <T> SinkFunction<T> createSink(String url, String username, String
password, String sql) {
        JdbcConnectionOptions connectionOptions = new
```

```

JdbcConnectionOptions.JdbcConnectionOptionsBuilder()
    .withDriverName(CLICKHOUSE_DRIVER_NAME)
    .withUrl(url)
    .withUsername(username)
    .withPassword(password)
    .build();

return JdbcSink.sink(
    sql,
    new ClickHousePreparedStatementSetter<T>(),
    new JdbcExecutionOptions.Builder().build(),
    connectionOptions);
}
}

```

此处创建了一个 **ClickHousePreparedStatementSetter** 类实例，该类需要实现 **JdbcStatementBuilder** 接口，并在这里进行反射处理，获取对象属性，忽略有 **TransientSink** 注解的属性：

```

import org.apache.flink.connector.jdbc.JdbcStatementBuilder;
import java.lang.reflect.Field;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class ClickHousePreparedStatementSetter<T> implements JdbcStatementBuilder<T> {
    @Override
    public void accept(PreparedStatement preparedStatement, T t) throws SQLException {
        Field[] fields = t.getClass().getDeclaredFields();
        int paramIndex = 1;
        for (Field field : fields) {
            field.setAccessible(true);
            if (field.getAnnotation(TransientSink.class) == null) { // 忽略有
TransientSink 注解的属性
                try {
                    Object value = field.get(t);

```

```

        preparedStatement.setObject(paramIndex, value);
        paramIndex++;
    } catch (IllegalAccessException e) {
        throw new RuntimeException("Error setting JDBC parameter", e);
    }
}
}
}
}
}
}
}
}
}

```

注意，这里假设你已经定义了 **TransientSink** 注解：

```

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface TransientSink {
}

```

这样，当你创建一个含有 **TransientSink** 注解的类，写入 ClickHouse 时，就会忽略这个注解的属性。

在使用时，只需要调用 **ClickHouseJdbcSinkFactory.createSink** 方法创建 sink，然后

反馈

内容没有生成完，让 GPT 继续

第 2 轮提示

提示

继续

答

然后将此 sink 添加到数据流中，如下所示：

```
import org.apache.flink.streaming.api.datastream.DataStream;

public class YourFlinkJob {
    public static void main(String[] args) throws Exception {
        // 创建 ExecutionEnvironment 和 DataStream 等...

        // 以下代码是示例，你可能需要根据实际情况修改
        String clickhouseUrl = "jdbc:clickhouse://localhost:8123/default";
        String clickhouseUsername = "default";
        String clickhousePassword = "";
        String sql = "INSERT INTO myTable(field1, field2) VALUES(?, ?)";
        SinkFunction<MyClass> clickhouseSink = ClickHouseJdbcSinkFactory.createSink(
            clickhouseUrl,
            clickhouseUsername,
            clickhousePassword,
            sql
        );

        DataStream<MyClass> dataStream = // 获取你的 DataStream, MyClass 是你的实体类

        // 添加 sink 到你的 dataStream
        dataStream.addSink(clickhouseSink);

        // 开始执行
        // env.execute("My Flink Job");
```

```
}  
}
```

在这个例子中，**MyClass** 是你要写入 ClickHouse 的数据类。假设它有两个属性 **field1** 和 **field2**，并且 **field2** 上有 **TransientSink** 注解，那么在写入 ClickHouse 时，**field2** 就会被忽略。

请注意，上述代码可能需要根据你的实际需求进行修改。我希望这个例子可以帮助你理解如何使用 Flink 的 JDBC connector 写入 ClickHouse，并忽略有 **TransientSink** 注解的属性。

反馈

可以看到 GPT 写出了正确的代码。接下来，我们还可以询问一些更细节的东西

第 3 轮提示

提示

这样写是一次写一条还是攒批写

答

在 Flink 的 JDBC connector 中，**JdbcSink** 类默认的行为是一次写入一行数据。然而，可以通过设置 **JdbcExecutionOptions** 对象来改变这一行为，实现一次批量写入多行数据。

在创建 **JdbcSink** 的时候，我们可以提供一个 **JdbcExecutionOptions** 对象，指定批量写入的大小。例如，以下代码将会创建一个每次写入 1000 行数据的 sink：

```
public static <T> SinkFunction<T> createSink(String url, String username, String password,  
String sql) {  
    JdbcConnectionOptions connectionOptions = new  
JdbcConnectionOptions.JdbcConnectionOptionsBuilder()
```

```
        .withDriverName(CLICKHOUSE_DRIVER_NAME)
        .withUrl(url)
        .withUsername(username)
        .withPassword(password)
        .build();

JdbcExecutionOptions executionOptions = new JdbcExecutionOptions.Builder()
    .withBatchSize(1000) // 每次写入 1000 行数据
    .build();

return JdbcSink.sink(
    sql,
    new ClickHousePreparedStatementSetter<T>(),
    executionOptions,
    connectionOptions);
}
```

这样，Flink 会将多行数据暂存起来，然后一次性写入到数据库，以提高效率。但请注意，这会增加延迟，因为 Flink 需要等待足够多的数据才能进行写入。根据你的具体需求以及 ClickHouse 的能力，你可能需要调整批量大小。

反馈

我们可以通过这种方式不停迭代，最终得到我们满意的代码。

习题二 编写接口 Demo

第 1 轮提示

提示

我现在要写一个 spring boot 程序，外部应用调我的接口，我就可以将 clickhouse 里的数据以 json 的方式返回给他们。可以帮帮我写一个 demo 吗。要求使用 java, spring boot, mybatis

答

好的，我可以提供一个简单的例子给你。首先，确保你已经在 Spring Boot 项目中配置了 MyBatis 和 Clickhouse 的连接。

假设你已经创建了一个 Spring Boot 应用，并添加了 Spring Boot Starter Web, Spring Boot MyBatis Starter, ClickHouse Driver 等相关依赖。

我假设你的项目的结构是这样的：

```
.
├── src
│   ├── main
│   │   ├── java
│   │   │   ├── com
│   │   │   │   ├── example
│   │   │   │   │   ├── demo
│   │   │   │   │   │   ├── DemoApplication.java
│   │   │   │   │   │   ├── controller
│   │   │   │   │   │   │   ├── DataController.java
│   │   │   │   │   │   ├── mapper
│   │   │   │   │   │   │   ├── DataMapper.java
│   │   │   │   │   │   ├── service
│   │   │   │   │   │   │   ├── DataService.java
```

```
| | | └─ impl
| | | └─ DataServiceImpl.java
| | └─ resources
| └─ application.properties
└─ test
└─ pom.xml
```

1. 添加必要的依赖:

```
<!-- pom.xml -->

<dependencies>
  <!-- Spring Boot Starter Web -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <!-- Spring Boot MyBatis Starter -->
  <dependency>
    <groupId>org.mybatis.spring.boot</groupId>
    <artifactId>mybatis-spring-boot-starter</artifactId>
    <version>2.2.0</version>
  </dependency>

  <!-- ClickHouse JDBC driver -->
  <dependency>
    <groupId>ru.yandex.clickhouse</groupId>
    <artifactId>clickhouse-jdbc</artifactId>
    <version>0.2.4</version>
  </dependency>
</dependencies>
```

2. 配置你的 application.properties:

```
# application.properties

spring.datasource.url=jdbc:clickhouse://<your-clickhouse-url>:8123/<database>
spring.datasource.driver-class-name=ru.yandex.clickhouse.ClickHouseDriver
spring.datasource.username=<username>
spring.datasource.password=<password>

mybatis.type-aliases-package=com.example.demo.mapper
mybatis.mapper-locations=classpath*:mapper/*.xml
```

3. 创建你的 Mapper 接口:

```
// com/example/demo/mapper/DataMapper.java

package com.example.demo.mapper;

import org.apache.ibatis.annotations.Mapper;
import org.apache.ibatis.annotations.Select;

import java.util.Map;

@Mapper
public interface DataMapper {

    @Select("SELECT * FROM your_table LIMIT #{limit}")
    Map<String, Object> selectData(int limit);
}
```

4. 创建你的 Service:

```
// com/example/demo/service/DataService.java
```

```
package com.example.demo.service;

import java.util.Map;

public interface DataService {
    Map<String, Object> getData(int limit);
}

// com/example/demo/service/impl/DataServiceImpl.java

package com.example.demo.service.impl;

import com.example.demo.mapper.DataMapper;
import com.example.demo.service.DataService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.Map;

@Service
public class DataServiceImpl implements DataService {

    @Autowired
    private DataMapper dataMapper;

    @Override
    public Map<String, Object> getData(int limit
```

反馈

给有写完，让 GPT 继续

第 2 轮提示

提示

继续

答

对不起，我在上面的回答中有一部分遗漏了，以下是完整的 `DataServiceImpl` 类：

```
// com/example/demo/service/impl/DataServiceImpl.java

package com.example.demo.service.impl;

import com.example.demo.mapper.DataMapper;
import com.example.demo.service.DataService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.Map;

@Service
public class DataServiceImpl implements DataService {

    @Autowired
    private DataMapper dataMapper;

    @Override
    public Map<String, Object> getData(int limit) {
        return dataMapper.selectData(limit);
    }
}
```

5. 创建你的 Controller

```
// com/example/demo/controller/DataController.java

package com.example.demo.controller;

import com.example.demo.service.DataService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import java.util.Map;

@RestController
public class DataController {

    @Autowired
    private DataService dataService;

    @GetMapping("/data")
    public Map<String, Object> getData(@RequestParam(defaultValue = "10") int limit) {
        return dataService.getData(limit);
    }
}
```

这个例子会提供一个 HTTP 接口，`/data`，当你请求这个接口的时候，会从你的 Clickhouse 数据库中获取数据，并以 JSON 格式返回。你可以通过 `limit` 参数来限制返回的行数。

请注意，你需要替换 `<your-clickhouse-url>`，`<database>`，`<username>`，`<password>`，`your_table` 等为你的实际值。并确保你的 mapper XML 文件中的 SQL 语句是正确的，可以获取到你需要的数据。

反馈

现在，你已经得到了一个基本的 spring boot 接口 demo。后面，你可以根据自己的需要，让 GPT 解释其中的细节，或者晚上接口中的一部分方法。